

What is the relaxation condition in graph theory

Asked 15 years, 2 months ago Modified 3 years, 5 months ago Viewed 11k times



I'm trying to understand the main concepts of graph theory and the algorithms within it. Most algorithms seem to contain a "Relaxation Condition" I'm unsure about what this is.

20



Could some one explain it to me please.

An example of this is dijkstras algorithm, here is the pseudo-code.



```
1 function Dijkstra(Graph, source):
2     for each vertex v in Graph:           // Initializations
3         dist[v] := infinity               // Unknown distance function from source to v
4         previous[v] := undefined          // Previous node in optimal path from source
5     dist[source] := 0                     // Distance from source to source
6     Q := the set of all nodes in Graph
7     // All nodes in the graph are unoptimized - thus are in Q
8     while Q is not empty:                 // The main loop
9         u := vertex in Q with smallest dist[]
10        if dist[u] = infinity:
11            break                          // all remaining vertices are inaccessible
12        from source
13        remove u from Q
14        for each neighbor v of u:          // where v has not yet been removed from Q.
15            alt := dist[u] + dist_between(u, v)
16            if alt < dist[v]:               // Relax (u,v,a)
17                dist[v] := alt
18                previous[v] := u
19    return dist[]
```

Thanks

[graph](#) [graph-theory](#) [conditional-statements](#)

[Share](#) [Improve this question](#) [Follow](#)

asked Apr 7, 2010 at 13:28



[alchemey89](#)

485 ● 2 ● 6 ● 8

3 Answers

Sorted by: [Highest score \(default\)](#)





43



Relaxation step:

- You have two nodes, u and v
- For every node, you have a *tentative distance* from the source node (for all nodes except for the source, it starts at positive infinity and it only decreases up to reaching its minimum).

The relaxation step basically is asking this:

- I already know that I can reach v with some path of distance $\text{dist}[v]$. Could I improve on this by going to v through u instead? (where the distance of the latter would be $\text{dist}[u] + \text{weight}(u, v)$)

Graphically:



You know some path $s \leadsto v$ which has distance $\text{dist}[v]$, and you know some path $s \leadsto u$ which has distance $\text{dist}[u]$. If $\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]$, then the path $s \leadsto u \rightarrow v$ is shorter than $s \leadsto v$, so you'd better use that one!

(I write $a \leadsto b$ to mean a path of *any* length from a to b , while $a \rightarrow b$ I mean a direct single edge from a to b).

You may also want to check this lecture: <http://ocw.mit.edu/OcwWeb/Electrical-Engineering-and-Computer-Science/6-046JFall-2005/VideoLectures/detail/embed17.htm>

Share Improve this answer Follow

edited Apr 7, 2010 at 16:09

answered Apr 7, 2010 at 13:47



Dimitris Andreou

8,898 ● 2 ● 34 ● 36

- 2 Glad to hear (hopefully I didn't confuse you with the typos, now fixed). You could also press the tick symbol on the left to mark this as the correct answer, if you think it is (I surely think it is :) – **Dimitris Andreou** Apr 7, 2010 at 16:09



13



One of the meanings of the english word "relaxation" is decreasing something. Because at lines 14,15 and 16 you are essentially checking if you can decrease(optimize) the currently computed distance, I guess that's why it is called "relaxation condition".

Share Improve this answer Follow

answered Apr 7, 2010 at 13:43



Petar Minchev

47.4k ● 12 ● 107 ● 120



One way to remember is:

0



So we say:

- . relax outgoing edges of closest vertex in Dijkstra's algorithm
- . relax edges repeatedly in Bellman-Ford algorithm



Both implying to try to reduce distance from start node to reach vertex which is on the other end of edge

[Share](#) [Improve this answer](#) [Follow](#)

answered Jan 16, 2022 at 12:40



Manohar Reddy Poreddy

27.7k ● 10 ● 168 ● 149

Start asking to get answers

Find the answer to your question by asking.

[Ask question](#)

Explore related questions

[graph](#) [graph-theory](#) [conditional-statements](#)

See similar questions with these tags.